Buffer Cache

Dr. Arati Nimgaonkar

Buffer cache…… Pre-requisite

- Student should have fundamental knowledge of computer operating system.
- Student should have knowledge of writing algorithm.
- Student should know data structure concept.

Buffer cache…… Objectives

At the end of this lecture student will be able to,

- Describe the use and need of the buffering mechanism.
- Draw and explain the buffer pool in which they will learn concept of hash queue and free list.
- List out and explain the buffer retrieval scenarios.

Buffer cache……

Important Concepts

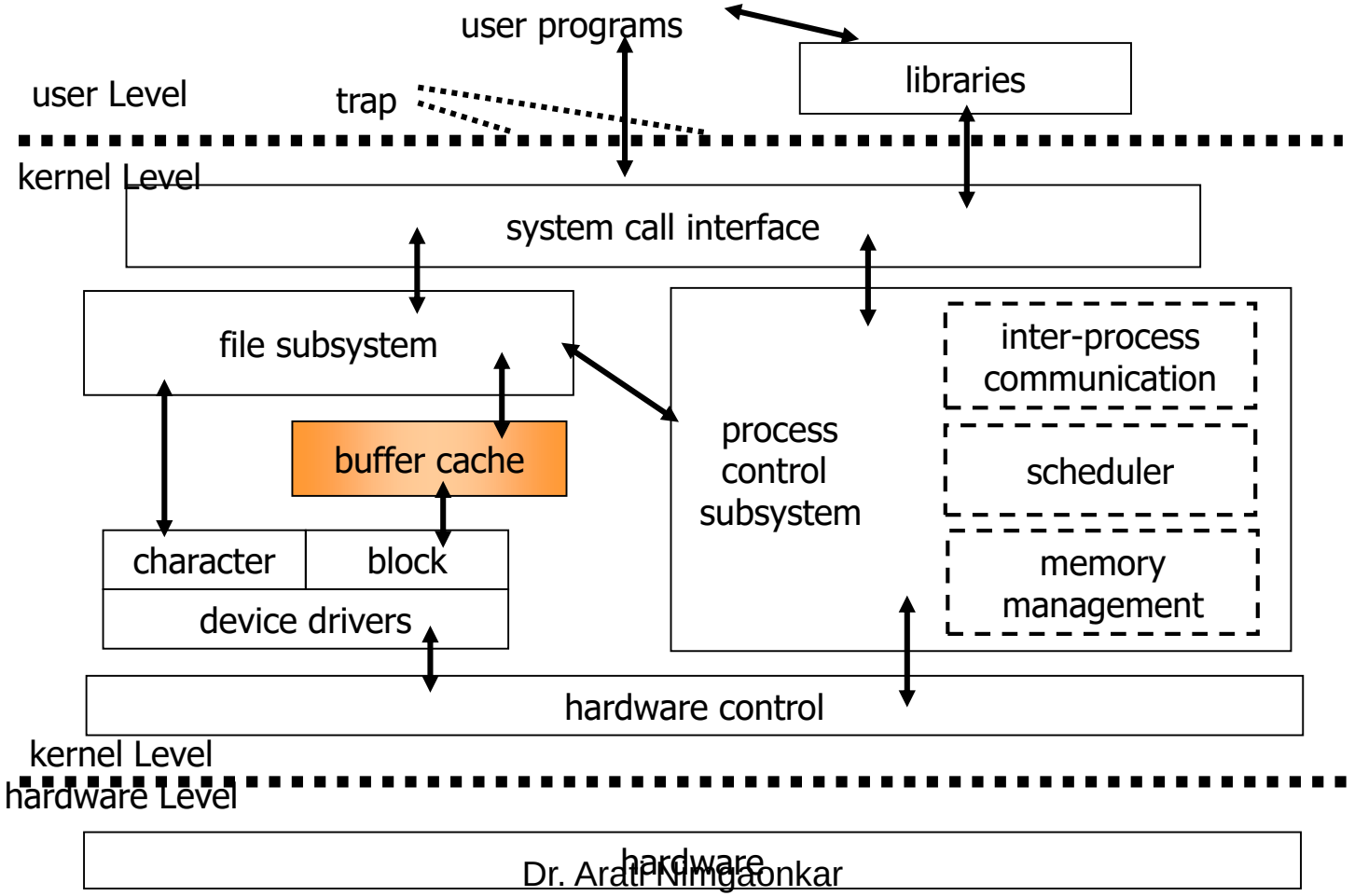
- Buffer header
- Buffer cache (Buffer pool)
- Hash queue
- Buffer free list
- Delayed write
- Old & new hash queue

Important Facts

- Block no.
- Device no. (File system no.)
- Hash function

Architecture of Unix—Buffer cache

- Buffering mechanism is used to Interact with block I/O device drivers to initiate data transfer to and from kernel.



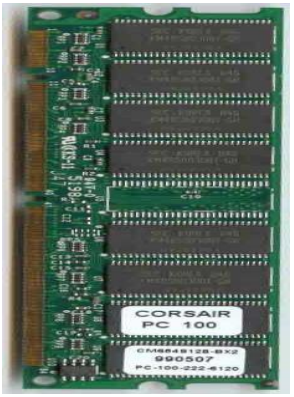
Accessing data from HDD

- When a process wants to access data from a file, the kernel brings the data into main memory, alters it and then request to save in the file system.



Bring data

Save data



Hard disk **Main Memory**

- In this case system performance decreases since kernel has to traverse from disk to main memory & main memory to disk to fetch the data.

The kernel therefore 'attempts to minimize the frequency of disk access by keeping a pool of internal data buffers, called the buffer cache.

What is a buffer??

A buffer, also called buffer memory, is a portion of a computer's memory that is set aside as a temporary holding place for data that is being sent to or received from an external device, such as a hard disk drive (HDD), keyboard or printer.

A buffer in a computer system is usually located between two devices that have different speeds for handling data or used when there is a difference in the timing of events.



Bring data for manipulation



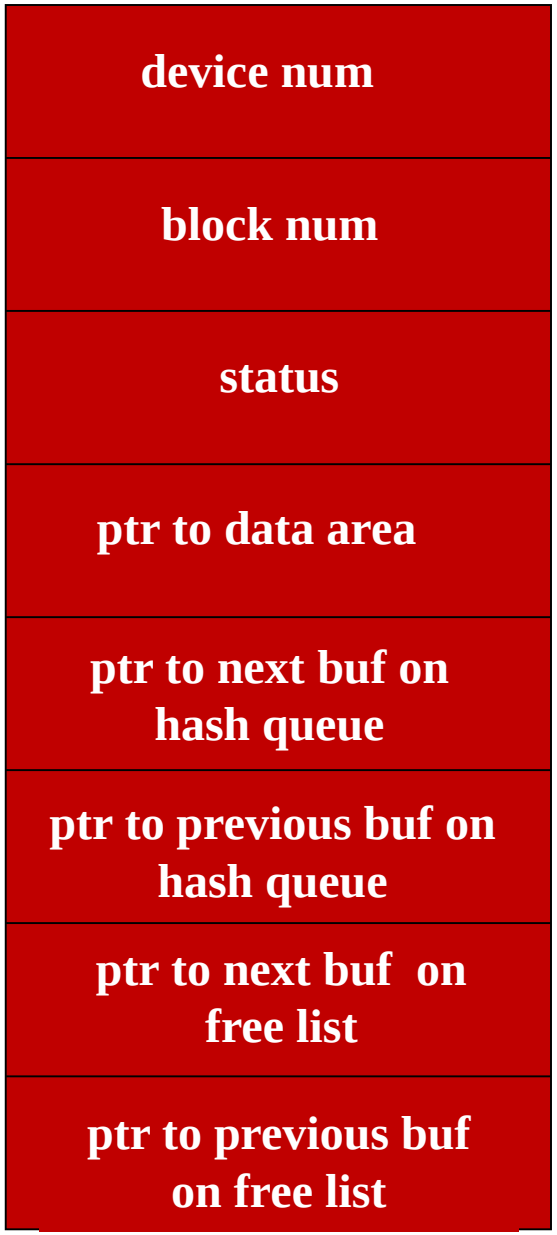
•To increase the performance & to reduce the traversing time of data to & from disk, temporary storage is used which contains the data from recently used disk blocks.

This temporary storage is called as buffer.

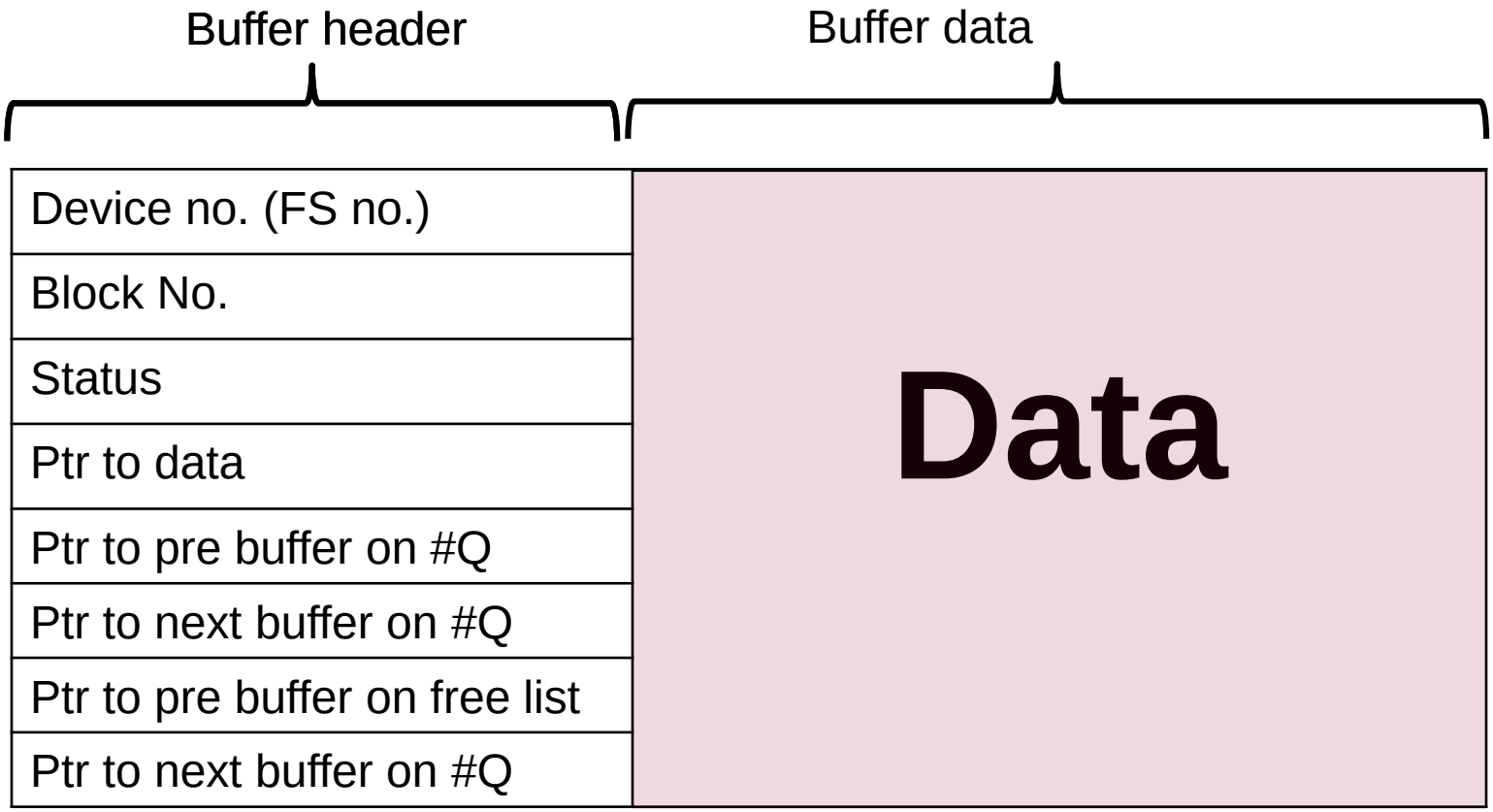
Buffer cache…… Buffer Header

- device num -logical file system number
- block num-block number of the data on disk
- Status-
  - The buffer is currently locked.
  - The buffer contains valid data.
  - Marked for delayed-write.
  - The kernel is currently reading or writing the contents of the disk.
  - A process is currently waiting for the buffer to become free.
- Various pointers maintaining the structure of the buffer pool.

But at the same time, a disk block cannot be mapped to multiple buffers.



Single buffer structure



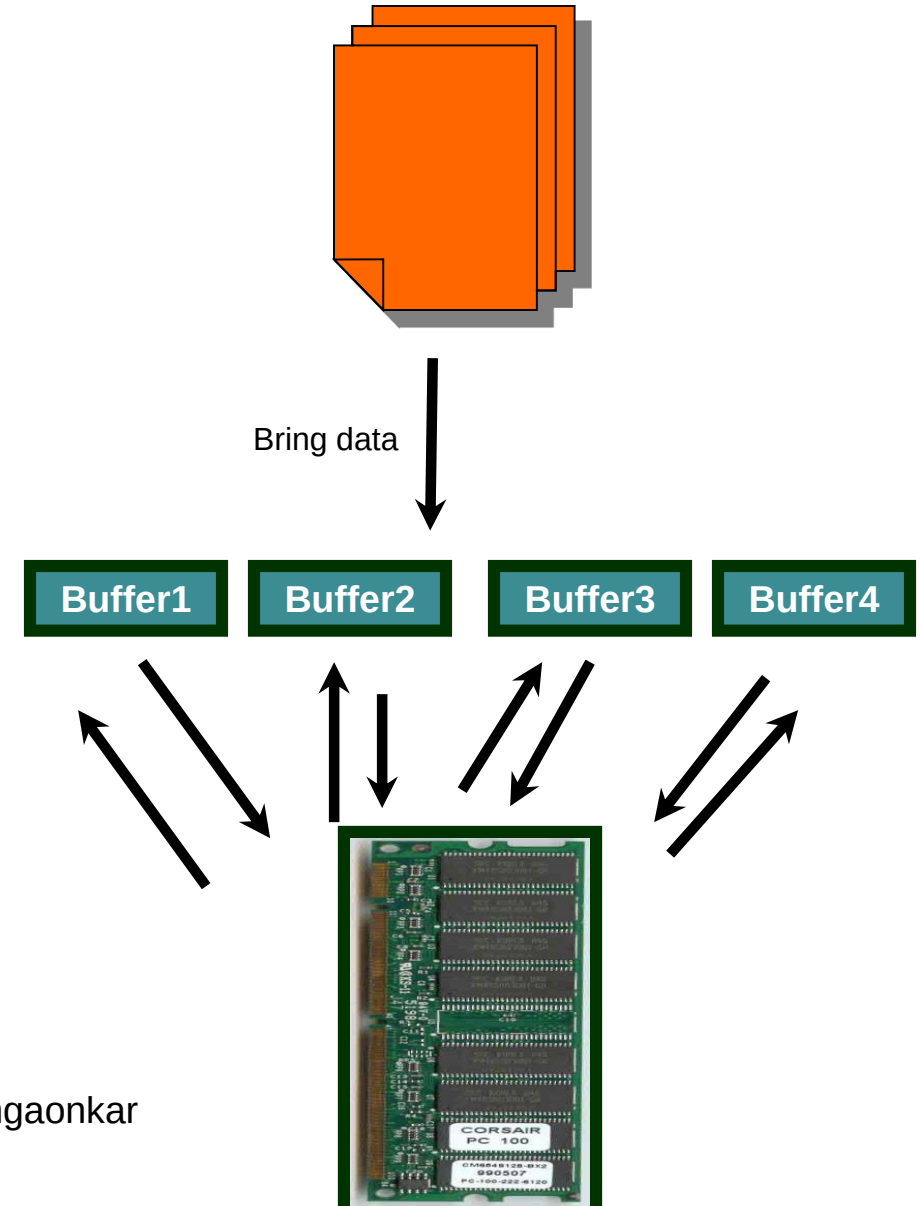
Buffer cache…… Buffer Header

```
struct buffer_head{
        /*First cache line: */
        struct buffer_head *b_next;          /*Hash queue list*/
        unsigned long b_blocknr;             /*block number*/
        unsigned long b_size;                /*block size*/
        kdev_t b_dev;                        /*device(B_FREE = free)*/
        kdev_t b_rdev;                       /*Read device*/
        unsigned long b_rsector;             /*Real Buffer location on disk*/
        struct buffer_head *b_this_page;     /*circular list of buffers in one page*/
        unsigned long b_state;               /*buffer state bitmap(see above)*/
        struct buffer_head *b_next_free;
        unsigned int  b_count;               /*users using this block*/
        char *b_data;                        /*pointer to data block(1024 bytes)*/
        unsigned int b_list;                 /*List that this buffer appears*/
        unsigned long b_flushtime;           /*Time when this(dirty) buffer should be written*/
        struct wait_queue *b_wait;
        struct buffer_head **b_pprev;        /*doubly linked list of hash-queue*/
        struct buffer_head *b_prev_free;     /* double linked list of buffers*/
        struct buffer_head *b_reqnext;       /*request queue*/

        /*I/O completion*/
        void (*b_end_io)(struct buffer_head *bh,  int uptodate);
        void *b_dev_id;
};
```

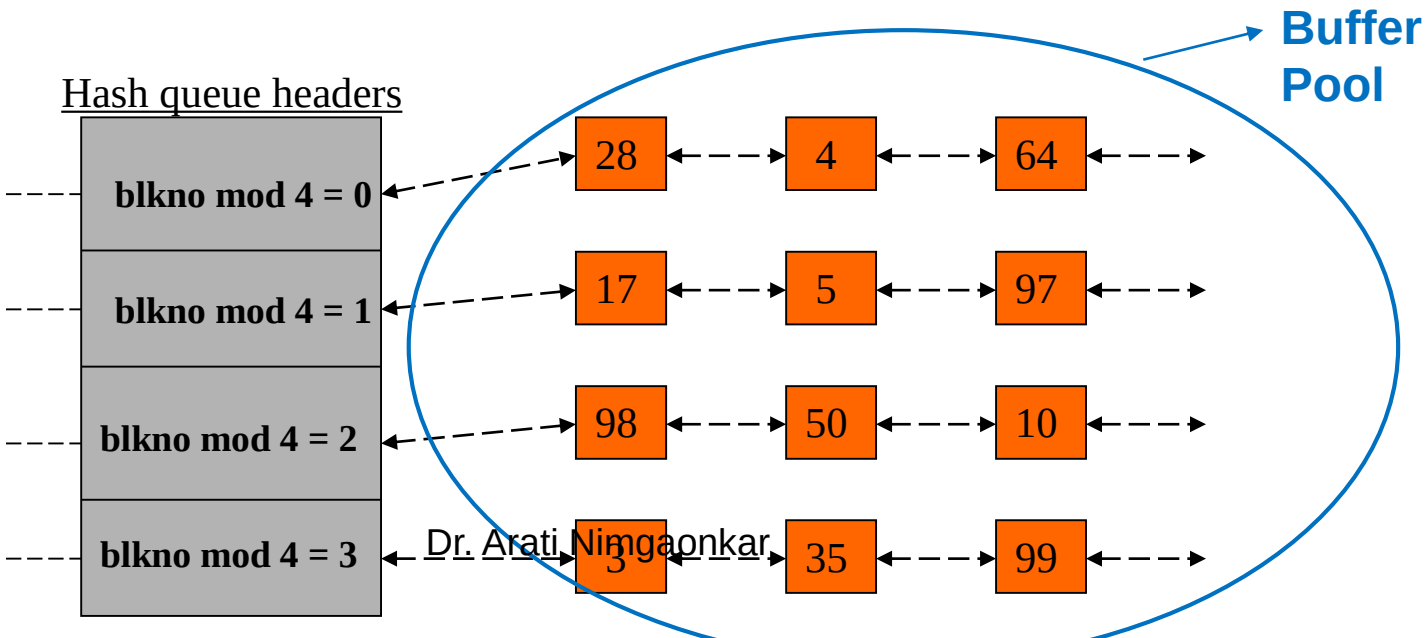
Buffer cache…… Buffer pool

- Not only one but collection of buffers are used. This collection of buffers is termed as buffer pool or buffer cache.
- When reading data from disk, the kernel attempts to read from buffer cache.
- If data is already in the buffer cache, the kernel does not need to read from disk
- If data is not in the buffer cache, the kernel reads the data from disk and cache it



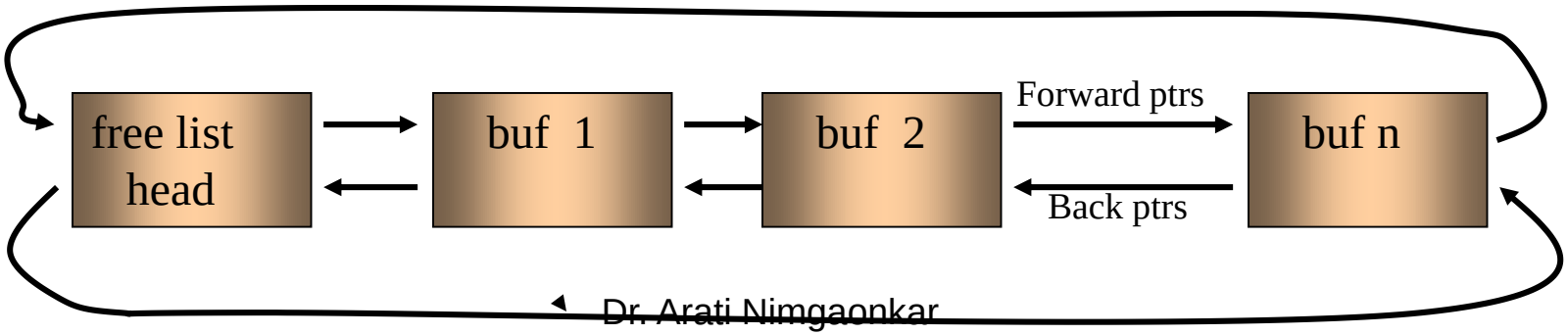
Buffer cache…… Hash queue

- When the kernel accesses a disk block-
  - Searches for a buffer with appropriate device-block combination.
  - Separate hash queues (circular *doubly linked list*) are maintained to avoid to search entire pool hashed as a function of the device and block num.
  - It distributes the buffers uniformly across the set of hash queue.
  - Every disk block exists on one and only on hash queue and only once on the queue



Buffer cache…… free list

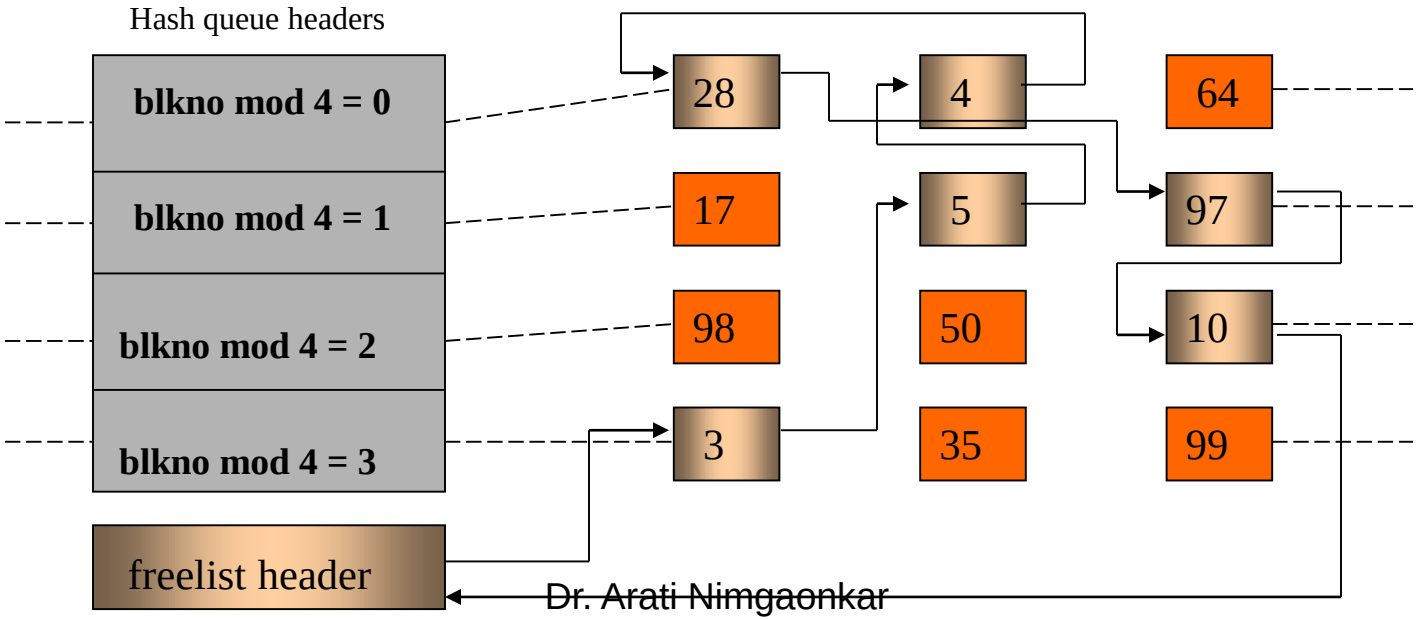
- Buffer pool is maintained according to *Least Recently Used algorithm*.
- The kernel maintains a free list of buffer using circular doubly linked list with dummy buffer header which marks it's beginning & ending.
- Take a buffer from the head of the free list. But can take from middle if it identifies a particular block.
- When returning kernel usually attaches the buffer to the tail, occasionally to the head of the list (error cases) but never to the middle.



Buffer cache…… Hash queue & free list

- Buffer may be simultaneously on a free list & on a hash queue provided status of the buffer is free.

- Kernel has two ways to find buffer….
  - -search a hash queue if searching for particular buffer.
  - -remove a buffer from free list if requires any buffer.

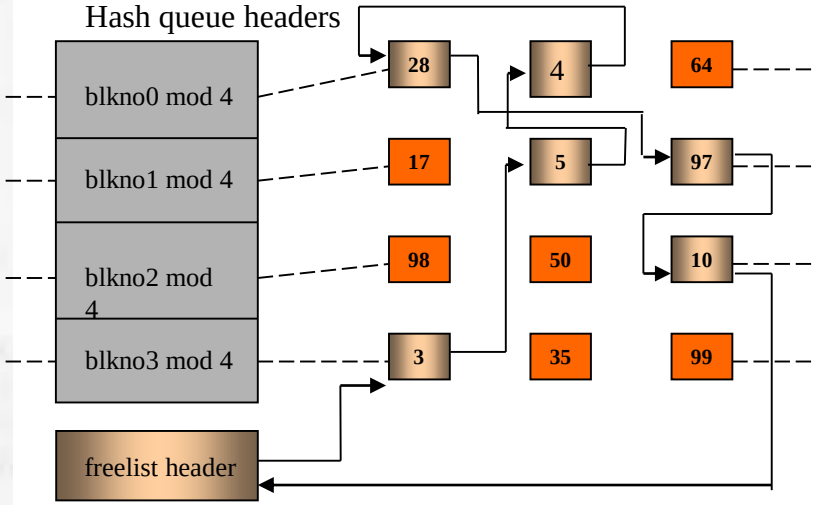


Buffer cache…… Algorithms

1. getblk – Allocation of a buffer from buffer pool
2. bread / breada – reading a block into a buffer
3. bwrite – writing contents of buffer to a disk block
4. brelse – releasing a buffer

Buffer cache…… getblk-Allocating buffer

```
algorithm getblk
input:   file system number
         block number
output: locked buffer that can now be used for block
{
        while (buffer not found)
        {
                if (block in hash queue)
                {
                        if (buffer busy)        /* scenario 5 */
                        {
                                sleep (event buffer becomes free);
                                continue;       /* back to while loop */
                        }
                        mark buffer busy;       /* scenario 1 */
                        remove buffer from free list;
                        return buffer;
                }
                else        /* block not on hash queue */
                {
                        if (there are no buffers on free list)        /* scenario 4 */
                        {
                                sleep (event any buffer becomes free);
                                continue;       /* back to while loop */
                        }
                        remove buffer from free list;
                        if (buffer marked for delayed write) {      /* scenario 3 */
                                asynchronous write buffer to disk;
                                continue;       /* back to while loop */
                        }
                        /* scenario 2 -- found a free buffer */
                        remove buffer from old hash queue;
                        put buffer onto new hash queue;
                        return buffer;
                }
        }
}
```



Buffer cache…… getblk-Allocating buffer

Scenarios (situations) arise during allocation of buffer.

1. The kernel finds the block on its hash queue, the buffer is free.
2. The kernel cannot find the block on the hash queue, so it allocates a buffer from free list
3. The kernel cannot find the block on the hash queue, and finds delayed write buffers on the hash queue.
4. The kernel cannot find the block on the hash queue, and finds free list empty
5. The kernel finds the block on its hash queue & the buffer is busy.

Buffer cache…… getblk - Ist Scenario

The kernel finds the block on its hash queue & the buffer is free.

Buffer cache……

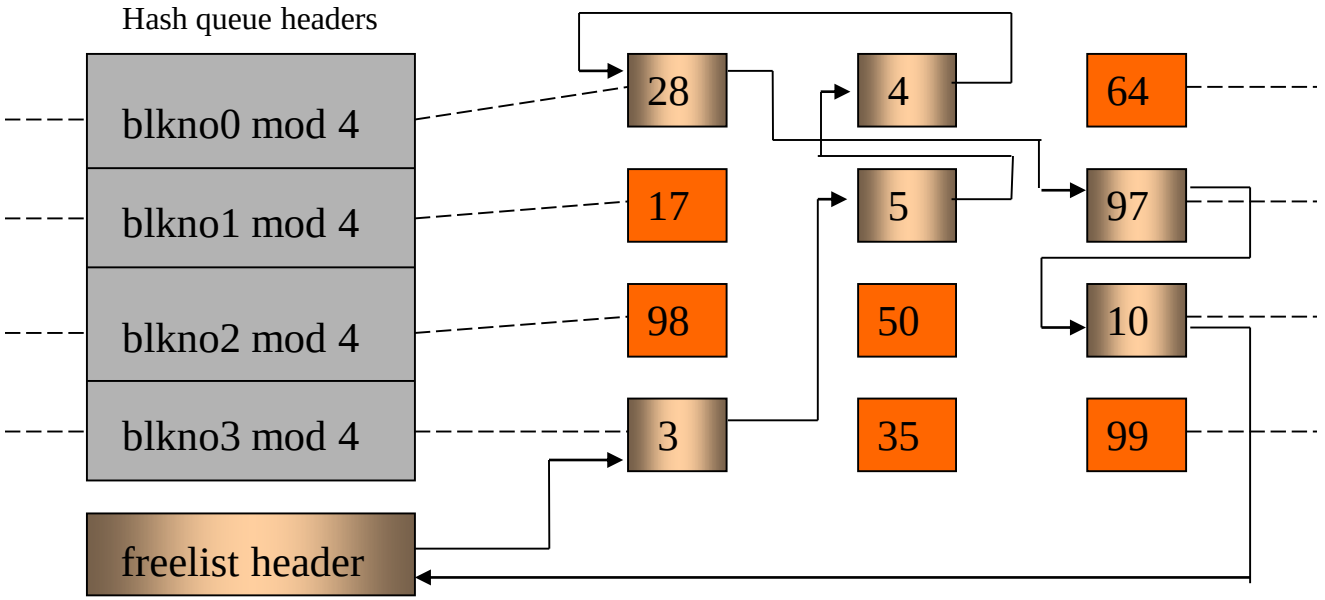
getblk - I[st] Scenario

The kernel finds the block on its hash queue & the buffer is free.

```
algorithm getblk
input:   file system number
         block number
output: locked buffer that can now be used for block
{
        while (buffer not found)
        {
                if (block in hash queue)
                {
                        if (buffer busy)        /* scenario 5 */
                        {
                                sleep (event buffer becomes free);
                                continue;       /* back to while loop */
                        }
                        mark buffer busy;       /* scenario 1 */
                        remove buffer from free list;
                        return buffer;
                }
                else        /* block not on hash queue */
                {
                        if (there are no buffers on free list)        /* scenario 4 */
                        {
                                sleep (event any buffer becomes free);
                                continue;       /* back to while loop */
                        }
                        remove buffer from free list;
                        if (buffer marked for delayed write) {      /* scenario 3 */
                                asynchronous write buffer to disk;
                                continue;       /* back to while loop */
                        }
                        /* scenario 2 -- found a free buffer */
                        remove buffer from old hash queue;
                        put buffer onto new hash queue;
                        return buffer;
                }
        }
}
```

Buffer cache…… getblk - Ist Scenario

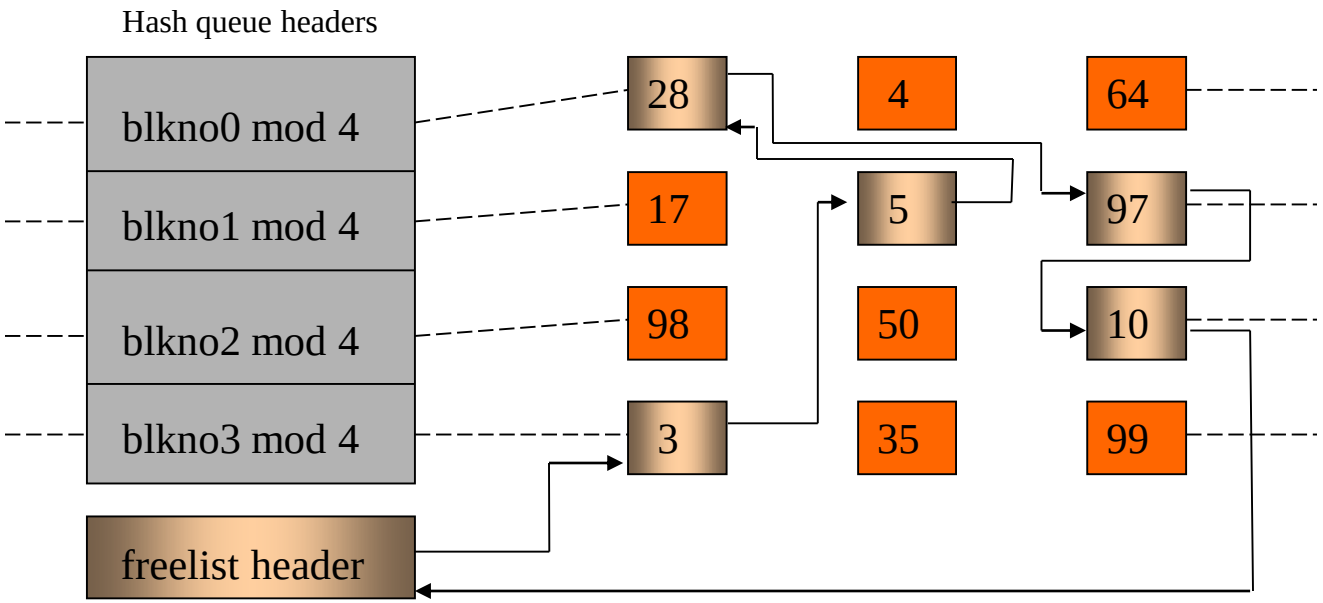
The kernel finds the block on its hash queue & the buffer is free.



Kernel Searches for block 4 & finds it on hash queue marked blkno0 mod 4 & also it is free

Buffer cache…… getblk - Ist Scenario

After removing block 4 from free list



Buffer cache…… getblk - IInd Scenario

The kernel cannot find the block on the hash queue, so it allocates a buffer from free list

Buffer cache……

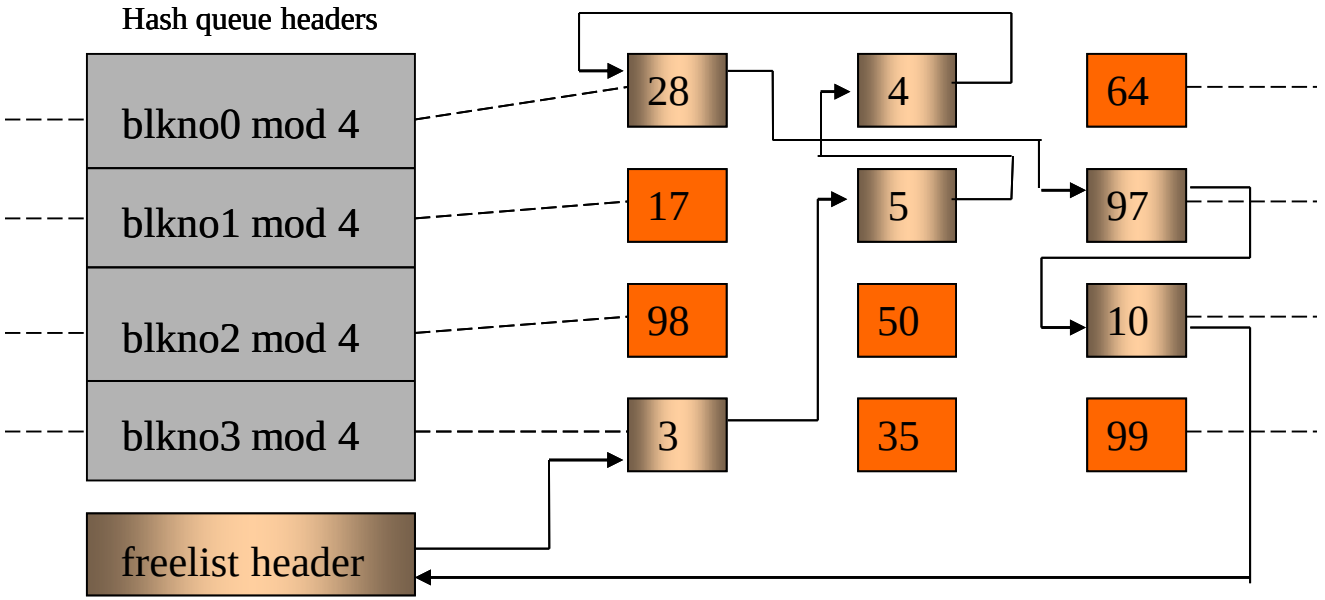
getblk – II[nd] Scenario

```
algorithm getblk
input:   file system number
         block number
output: locked buffer that can now be used for block
{
    while (buffer not found)
    {
        if (block in hash queue)
        {
            if (buffer busy)        /* scenario 5 */
            {
                sleep (event buffer becomes free);
                continue;       /* back to while loop */
            }
            mark buffer busy;       /* scenario 1 */
            remove buffer from free list;
            return buffer;
        }
        else      /* block not on hash queue */
        {
            if (there are no buffers on free list)      /* scenario 4 */
            {
                sleep (event any buffer becomes free);
                continue;       /* back to while loop */
            }
            remove buffer from free list;
            if (buffer marked for delayed write) {      /* scenario 3 */
                asynchronous write buffer to disk;
                continue;       /* back to while loop */
            }
            /* scenario 2 -- found a free buffer */
            remove buffer from old hash queue;
            put buffer onto new hash queue;
            return buffer;
        }
    }
}
```

The kernel cannot find the block on the hash queue, so it allocates a buffer from free list .

Buffer cache…… getblk - IInd Scenario

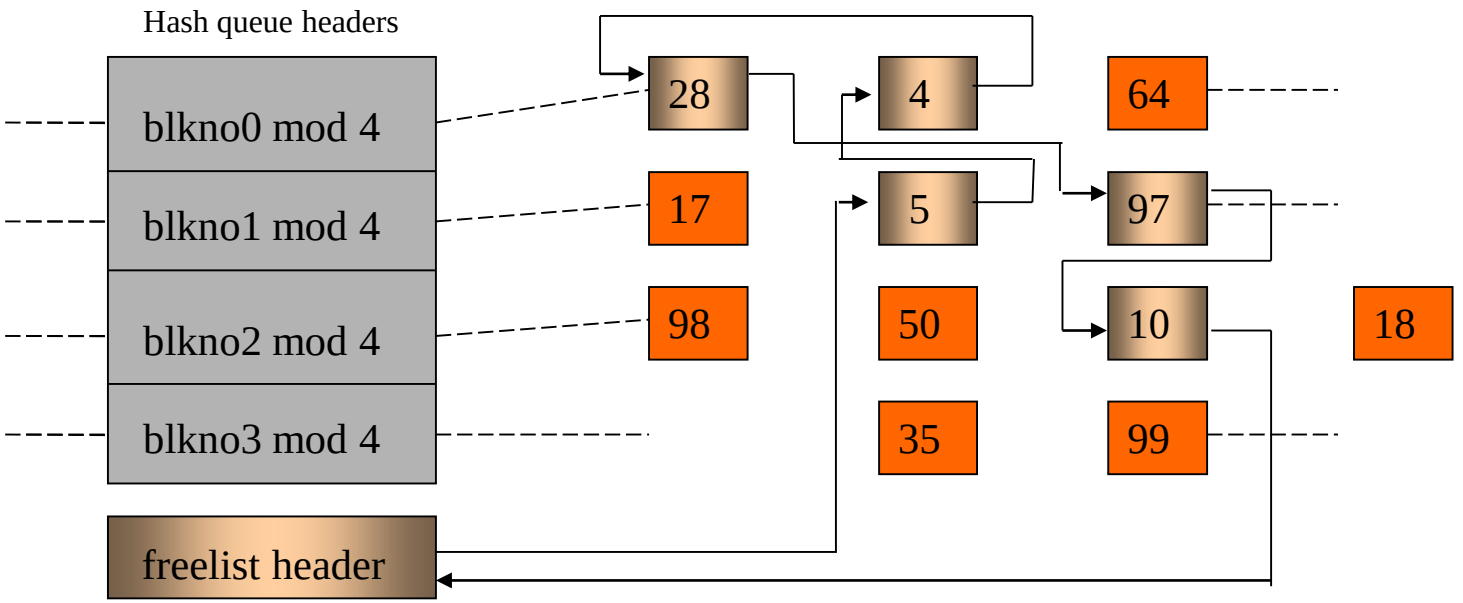
The kernel cannot find the block on the hash queue, so it allocates a buffer from free list



Kernel Searches for block 18 & can't find it on hash queue

Buffer cache…… getblk - IInd Scenario

Allocates first buffer from a free list, reassign the block & put it on appropriate hash queue.



Buffer cache……

getblk – III[rd] Scenario

```
algorithm getblk
input:   file system number
         block number
output: locked buffer that can now be used for block
{
    while (buffer not found)
    {
        if (block in hash queue)
        {
            if (buffer busy)        /* scenario 5 */
            {
                sleep (event buffer becomes free);
                continue;       /* back to while loop */
            }
            mark buffer busy;       /* scenario 1 */
            remove buffer from free list;
            return buffer;
        }
        else      /* block not on hash queue */
        {
            if (there are no buffers on free list)      /* scenario 4 */
            {
                sleep (event any buffer becomes free);
                continue;       /* back to while loop */
            }
            remove buffer from free list;
            if (buffer marked for delayed write) {      /* scenario 3 */
                asynchronous write buffer to disk;
                continue;       /* back to while loop */
            }
            /* scenario 2 -- found a free buffer */
            remove buffer from old hash queue;
            put buffer onto new hash queue;
            return buffer;
        }
    }
}
```

The kernel cannot find the block on the hash queue, and finds delayed write buffers on the hash queue.

Buffer cache…… getblk - IIIrd Scenario

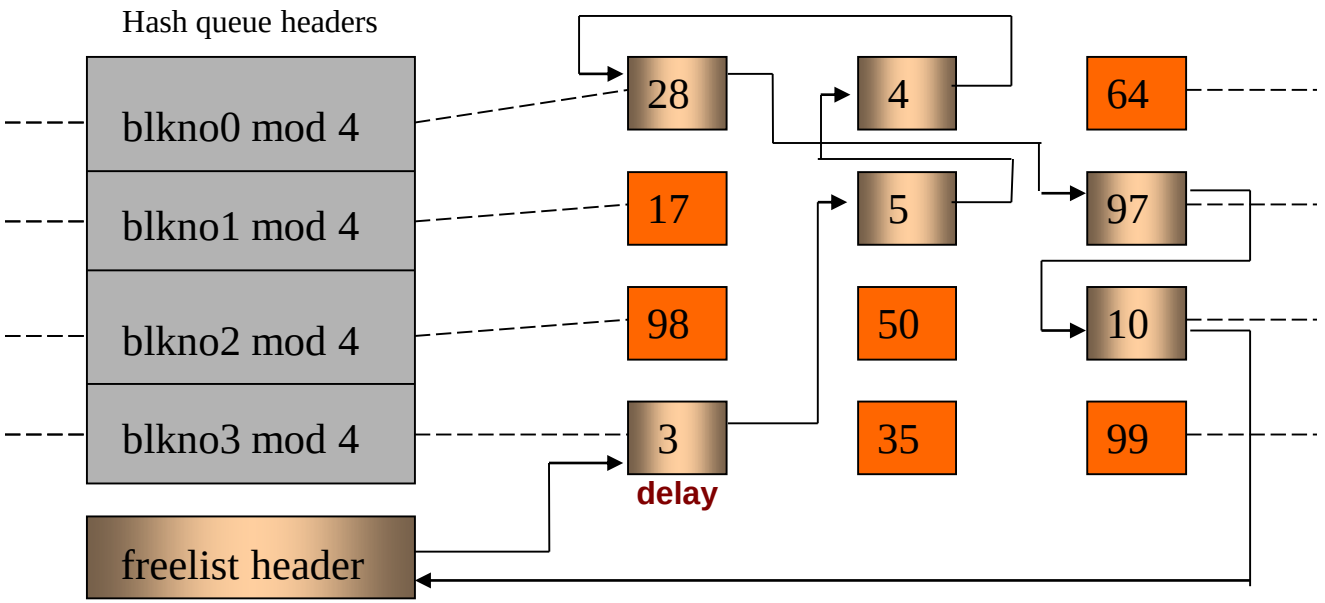
The kernel cannot find the block on the hash queue, and finds delayed write buffers on the hash queue.

What is delayed write in Unix?

- When the kernel writes data to a buffer, it marks it as delayed-write.
- This means that the buffer must be written to disk before the buffer can be re-used.
- Writing data to the buffer cache allows multiple updates to occur in memory rather than having to access the disk each time.

Buffer cache…… getblk - III[rd] Scenario

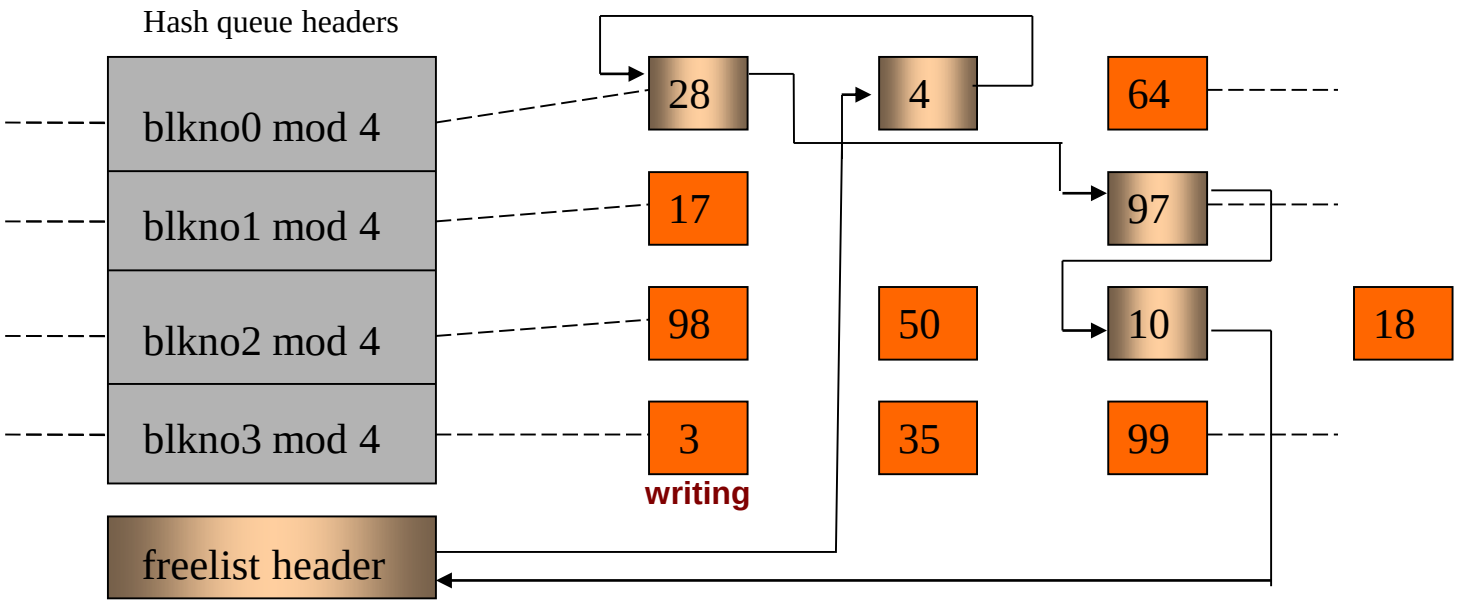
The kernel cannot find the block on the hash queue, and finds delayed write buffers on the hash queue.



Kernel Searches for block 18 & can't find it on hash queue. So try to allocate from free list.

Buffer cache…… getblk - IIIrd Scenario

Since buffer is marked for delayed write, it allocates next free buffer from list.



Reassigns block no., put it on appropriate hash queue & simultaneously writes delayed write buffer onto the disk.

Buffer cache…… getblk - IVth Scenario

The kernel cannot find the block on the hash queue, and finds free list empty.

Buffer cache……

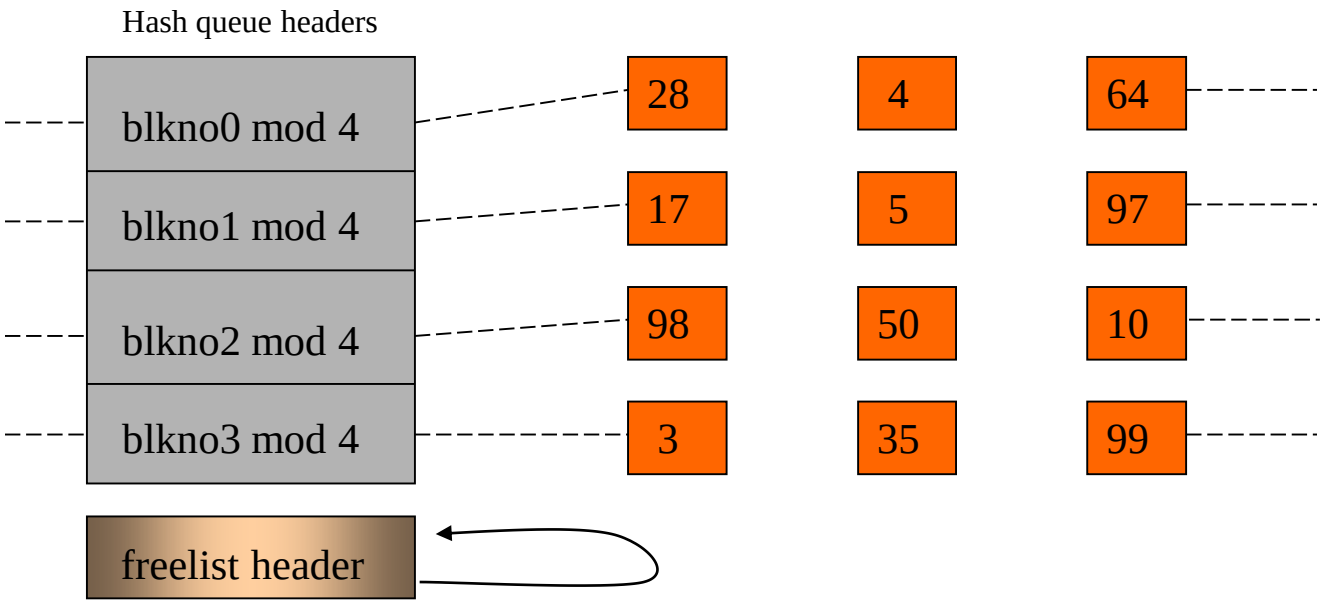
getblk – IVth Scenario

The kernel cannot find the block on the hash queue, and finds free list empty.

```
algorithm getblk
input:   file system number
         block number
output: locked buffer that can now be used for block
{
        while (buffer not found)
        {
                if (block in hash queue)
                {
                        if (buffer busy)        /* scenario 5 */
                        {
                                sleep (event buffer becomes free);
                                continue;       /* back to while loop */
                        }
                        mark buffer busy;       /* scenario 1 */
                        remove buffer from free list;
                        return buffer;
                }
                else        /* block not on hash queue */
                {
                        if (there are no buffers on free list)        /* scenario 4 */
                        {
                                sleep (event any buffer becomes free);
                                continue;       /* back to while loop */
                        }
                        remove buffer from free list;
                        if (buffer marked for delayed write) {      /* scenario 3 */
                                asynchronous write buffer to disk;
                                continue;       /* back to while loop */
                        }
                        /* scenario 2 -- found a free buffer */
                        remove buffer from old hash queue;
                        put buffer onto new hash queue;
                        return buffer;
                }
        }
}
```

Buffer cache…… getblk - IVth Scenario

The kernel cannot find the block on the hash queue, and finds free list empty.



In this situation, process goes, to sleep state till any of the buffer gets free.

Buffer cache…… getblk - Vth Scenario

The kernel finds the block on its hash queue & the buffer is busy.

Buffer cache……

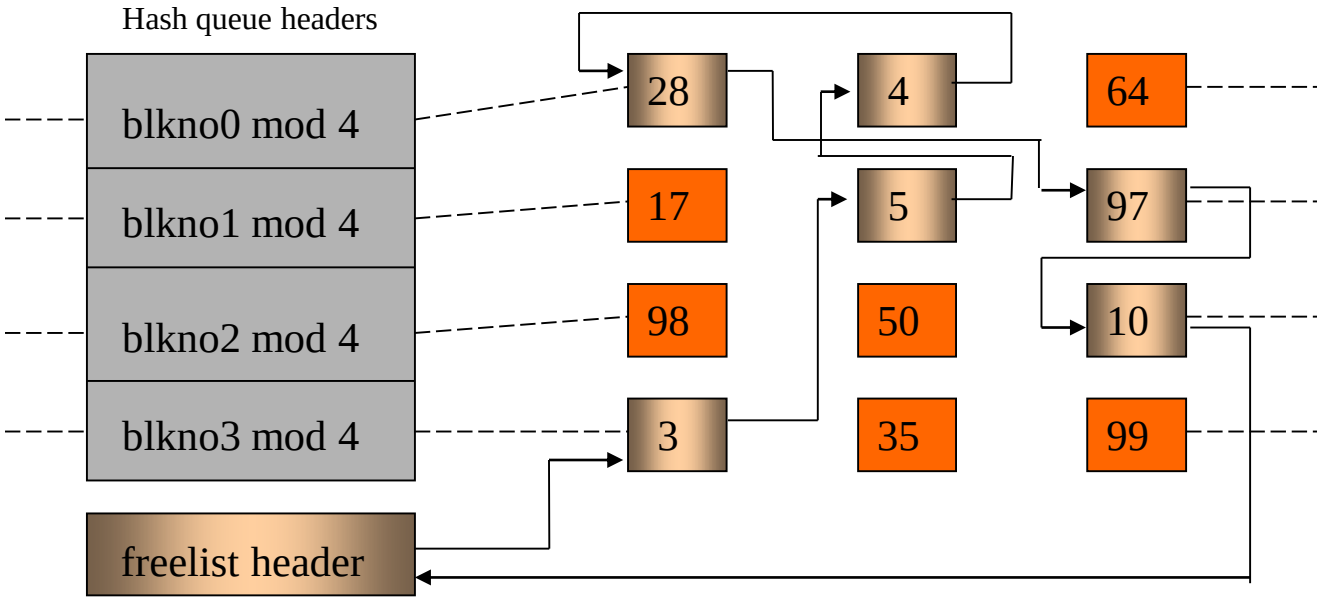
getblk – IVth Scenario

The kernel finds the block on its hash queue & the buffer is busy.

```
algorithm getblk
input:   file system number
         block number
output: locked buffer that can now be used for block
{
    while (buffer not found)
    {
        if (block in hash queue)
        {
            if (buffer busy)        /* scenario 5 */
            {
                sleep (event buffer becomes free);
                continue;       /* back to while loop */
            }
            mark buffer busy;       /* scenario 1 */
            remove buffer from free list;
            return buffer;
        }
        else      /* block not on hash queue */
        {
            if (there are no buffers on free list)       /* scenario 4 */
            {
                sleep (event any buffer becomes free);
                continue;       /* back to while loop */
            }
            remove buffer from free list;
            if (buffer marked for delayed write) {      /* scenario 3 */
                asynchronous write buffer to disk;
                continue;       /* back to while loop */
            }
            /* scenario 2 -- found a free buffer */
            remove buffer from old hash queue;
            put buffer onto new hash queue;
            return buffer;
        }
    }
}
```

Buffer cache…… getblk - Vth Scenario

The kernel finds the block on its hash queue & the buffer is busy.



Kernel Searches for block 99 & finds it on hash queue marked blkno3 mod 4 but it is marked as busy. So goes to sleep until it gets free.

Buffer cache…… getblk-Allocating buffer

```
algorithm getblk
input:   file system number
         block number
output: locked buffer that can now be used for block
{
      while (buffer not found)
      {
            if (block in hash queue)
            {
                  if (buffer busy)       /* scenario 5 */
                  {
                        sleep (event buffer becomes free);
                        continue;       /* back to while loop */
                  }
                  mark buffer busy;       /* scenario 1 */
                  remove buffer from free list;
                  return buffer;
            }
            else      /* block not on hash queue */
            {
                  if (there are no buffers on free list)       /* scenario 4 */
                  {
                        sleep (event any buffer becomes free);
                        continue;       /* back to while loop */
                  }
                  remove buffer from free list;
                  if (buffer marked for delayed write) {      /* scenario 3 */
                        asynchronous write buffer to disk;
                        continue;       /* back to while loop */
                  }
                  /* scenario 2 -- found a free buffer */
                  remove buffer from old hash queue;
                  put buffer onto new hash queue;
                  return buffer;
            }
      }
}
```

Buffer cache…… brelse – Releasing the buffer

Buffer cache…… brelse – Releasing the buffer

brelse (locked buffer)

wakeup all process event, waiting for any buffer to become free;
wakeup all process event, waiting for this buffer to become free;

raise processor execution level to block interrupts;

if (buffer is not old)
    put buffer at the end of free list
else *//buffer is old (Buffer contents are old only if it is marked as "delayed write")*
    put buffer at the beginning of free list

lower processor execution level to allow interrupt
unlock (buffer)

Buffer cache…… bread – Reading a block

```
algorithm bread /*block bread*/
input: file system block number
output: buffer containing data
{
    get buffer for block(algorithm getblk);
    if(buffer data valid)              // i.e. buffer is in a cache
        return buffer;
    initiate disk read;
    sleep(event disk read complete);
    return(buffer);
}
```

1. Kernel calls disk driver to schedule a read request & goes to sleep
2. Disk driver notifies the disk controller hardware that it wants to read data
3. Disk controller later transmits the data to the buffer.
4. Controller then interrupts the processor when I/O is complete.

Buffer cache…… breada – Reading a block ahead

- Higher level kernel module may anticipate that there is a need for a second disk block when process reads a file sequentially.
- So module requests the second I/O asynchronously in the hope that data will be in memory when needed, improving the performance.
- To do this, kernel executes **breada algorithm**

Buffer cache…… breada – Reading a block ahead

```
Algorithm breada
{
    if(first block not in cache)
    {
        get buffer for first block(algorithm getblk);
        if(buffer data not valid)
            initiate disk read;
    }
    if(second block not in cache)
    {
        get buffer for second block(algorithm getblk);
        if(buffer data valid)
            release buffer(algorithm brelse);
        else
            initiate disk read;
    }

    if(first block was originally in cache)
    {
        read first block(algorithm bread);
        return buffer;
    }
    sleep(event first buffer contains valid data);
    return buffer;
}
```

Buffer cache…… bwrite – Writing a disk block

```
algorithm bwrite    /* block write */
input:  buffer
output: none
{
        initiate disk write;
        if (I/O synchronous)
        {
                sleep(event I/O complete);
                release buffer (algorithm brelse);
        }
        else if (buffer marked for delayed write)
                mark buffer to put at head of free list;
}
```

1. The kernel informs the disk driver that it has a buffer whose contents should be output.
2. The disk driver schedules the block for I/O.
3. If the write is synchronous, the calling process goes the sleep awaiting I/O completion and releases the buffer when awakens.
4. If the write is asynchronous, the kernel starts the disk write. The kernel release the buffer when the I/O completes.

Buffer cache…… Advantages

1. The use of buffers allows uniform disk access, because the kernel does not need to know the reason for the I/O.

2. Since the kernel aligns data internally i.e. It places no data alignment restrictions on user processes doing I/O. So user processes don't have to worry about that.

3. Reduces amount of disk traffic, thereby increasing system throughput & decreasing response time.

4. Buffer algorithms help to insure file system integrity because they maintain a common, single image of disk blocks contained in the cache.